

Contents

KON414E - Team 14: 3D SLAM and Autonomous Navigation	3
Final Project Technical Report	3
Table of Contents	3
1. Project Overview	3
1.1 Project Requirements	3
1.2 System Architecture	3
1.3 Robot Specifications	4
1.4 Sensor Specifications	4
RGBD Camera	4
IMU	5
Ground Truth	5
2. Step 1: EKF Sensor Fusion	5
2.1 Objective	5
2.2 Implementation	5
EKF Parameters	5
Sensor Configuration	5
Process Noise Covariance (Diagonal)	6
2.3 Results	6
EKF Performance vs Ground Truth	6
Odometry Comparison	6
3. Step 2: Depth to PointCloud2 Conversion	6
3.1 Objective	6
3.2 Implementation	6
3.3 Published Topics	7
3.4 Additional Conversion: Depth to LaserScan	7
4. Step 3: 3D SLAM - Visual Mode	7
4.1 Objective	7
4.2 RTAB-Map Visual SLAM Configuration	8
Feature Detection	8
Loop Closure Parameters	8
Graph Optimization	8
Occupancy Grid	8
4.3 Visual SLAM Results	8
Map Quality Metrics	8
Localization Accuracy	9
5. Step 4: 3D SLAM - ICP Mode	9
5.1 Objective	9
5.2 RTAB-Map ICP Configuration	9
ICP Parameters	9
Optimization	9
5.3 ICP SLAM Results	9
Map Quality Metrics	9
Localization Accuracy	10
6. Step 5: Localization Performance Comparison	10
6.1 Objective	10
6.2 Evaluation Methodology	10
Metrics Calculated	10
Comparison Sources	10
6.3 Quantitative Comparison	10

EKF vs Ground Truth	10
SLAM Methods vs Ground Truth	10
Improvement Analysis	11
6.4 Qualitative Analysis	11
Visual SLAM Characteristics	11
ICP SLAM Characteristics	11
Key Findings	11
7. Step 6: 3D Mapping Performance Comparison	11
7.1 Objective	11
7.2 Evaluation Methodology	11
Metrics Calculated	11
7.3 Quantitative Comparison	12
Map Size and Coverage	12
Point Cloud Density	12
Bounding Box Dimensions	12
7.4 Mapping Progression Over Time (Visual SLAM)	12
7.5 2D Occupancy Grid Output	12
7.6 Qualitative Analysis	13
Visual SLAM Mapping	13
ICP SLAM Mapping	13
Recommendation	13
8. Step 7: Autonomous Navigation with Nav2	13
8.1 Objective	13
8.2 System Components	13
8.2.1 Hybrid SLAM Controller	13
Keyboard Controls	13
5-Region Obstacle Detection	14
8.2.2 Nav2 Configuration	14
8.3 Navigation Workflow	14
8.4 Launch Commands	15
8.5 Navigation Results	15
Successful Navigation Metrics	15
Integration with SLAM	15
9. Implementation Challenges and Solutions	16
9.1 Robot Physics Issues	16
Problem: Robot Falling Through Ground	16
9.2 SLAM Stability Issues	16
Problem: Map Drift and Reset During SLAM	16
9.3 Controller Conflicts	16
Problem: Hybrid Controller Conflicts with Nav2	16
9.4 Navigation Planning Failures	16
Problem: “Failed to create plan” Errors	16
9.5 Database Corruption	16
Problem: RTAB-Map database corrupted after system crash	16
10. Conclusions	17
10.1 Requirements Completion Summary	17
10.2 Key Findings	17
10.3 Future Improvements	17
Appendix A: File Locations	17
Appendix B: Commands Reference	17
Appendix C: Data Collection Summary	18

KON414E - Team 14: 3D SLAM and Autonomous Navigation

Final Project Technical Report

Course: KON414E - Principles of Robot Autonomy **Team:** Ceylan Tolunay, Atakan Yaman, Eren Yucetürk
Robot: Pioneer 3-DX (Differential Drive) **Environment:** Clearpath Robotics Office World (Gazebo Simulation)
Date: December 2024

Table of Contents

1. Project Overview
 2. Step 1: EKF Sensor Fusion
 3. Step 2: Depth to PointCloud2 Conversion
 4. Step 3: 3D SLAM - Visual Mode (RTAB-Map RGBD)
 5. Step 4: 3D SLAM - ICP Mode (RTAB-Map ICP)
 6. Step 5: Localization Performance Comparison
 7. Step 6: 3D Mapping Performance Comparison
 8. Step 7: Autonomous Navigation with Nav2
 9. Implementation Challenges and Solutions
 10. Conclusions
-

1. Project Overview

1.1 Project Requirements

Step	Requirement	Status	Implementation
1	EKF Sensor Fusion (IMU + Wheel Odometry)	COMPLETE	robot_localization package
2	Depth to PointCloud2 Conversion	COMPLETE	Gazebo RGBD plugin
3	3D SLAM (faster_lio or similar)	COMPLETE	RTAB-Map Visual SLAM
4	3D SLAM (fast_lio or similar)	COMPLETE	RTAB-Map ICP SLAM
5	Localization Comparison vs Ground Truth	COMPLETE	evaluation_node.py
6	3D Mapping Performance Comparison	COMPLETE	map_metrics.py
7	2D Projection + Autonomous Navigation	COMPLETE	Nav2 + RTAB-Map Grid

Note: RTAB-Map was used instead of FAST-LIO/Faster-LIO packages because: 1. RTAB-Map provides both Visual (RGBD) and ICP-based SLAM modes 2. Native ROS 2 Humble support without additional porting 3. Integrated 2D occupancy grid generation for Nav2 4. Proven performance with RGB-D cameras

1.2 System Architecture

SYSTEM ARCHITECTURE

```

Gazebo      Wheel
Simulation  >  Odometry      >
              /odom        EKF Filter
                        (robot_localization)

                        /odometry/filtered
IMU          >
> /imu/data

RGBD Camera  Depth to
depth/image  >  PointCloud
rgb/image    >  /depth/pts  >  RTAB-Map SLAM
                        (Visual or ICP)

                        /map (2D Grid)
                        /rtabmap/cloud_map

Nav2
- Global Planner
- Local Planner
PyGame      >  /cmd_vel  <  - Behavior Server
Controller  (Hybrid)

```

1.3 Robot Specifications

Component	Specification
Platform	Pioneer 3-DX
Drive Type	Differential Drive
Wheel Separation	0.30 m
Wheel Diameter	0.18 m
Chassis Mass	9.0 kg
Max Velocity	1.2 m/s
Max Angular Velocity	2.0 rad/s

1.4 Sensor Specifications

RGBD Camera

Parameter	Value
Field of View	90° (1.5708 rad)
Resolution	640 x 480 pixels
Depth Range	0.1 - 4.0 m
Update Rate	30 Hz
Image Format	R8G8B8
Mounting Position	Front, (0.15, 0, 0.30) m

IMU

Parameter	Value
Update Rate	100 Hz
Angular Velocity Noise	0.001745 rad/s ($\pm 0.1^\circ$ /s drift)
Linear Acceleration Noise	0.01 m/s ²
Mounting Position	Center, (0, 0, 0.20) m

Ground Truth

Parameter	Value
Plugin	libgazebo_ros_p3d.so
Topic	/ground_truth/odom
Update Rate	50 Hz
Noise	0 (perfect)

2. Step 1: EKF Sensor Fusion

2.1 Objective

Fuse IMU and wheel odometry data using the `robot_localization` package to achieve accurate state estimation.

2.2 Implementation

Configuration File: `src/robot_project/config/robot_localization.yaml`

EKF Parameters

```
ekf_filter_node:
  ros__parameters:
    frequency: 50.0           # Filter update rate (Hz)
    two_d_mode: true          # 2D constraint for ground robot

    # Coordinate frames
    map_frame: map
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom

    publish_tf: true
```

Sensor Configuration

Wheel Odometry (/odom):

```
odom0: /odom
odom0_config: [true, true, false, # x, y position
               false, false, true, # yaw orientation
               true, true, false, # vx, vy velocity
               false, false, true, # vyaw angular velocity
               false, false, false] # acceleration
```

IMU (/imu/data):

```
imu0: /imu/data
imu0_config: [false, false, false, # position (not from IMU)
              true,  true,  true,  # roll, pitch, yaw
              false, false, false, # velocity (not from IMU)
              true,  true,  true,  # angular velocity
              true,  true,  true]  # linear acceleration
imu0_remove_gravitational_acceleration: true
```

Process Noise Covariance (Diagonal)

State	Variance	Description
x, y	0.05	Position uncertainty
z	0.06	Vertical (constrained in 2D mode)
roll, pitch	0.03	Tilt angles
yaw	0.06	Heading
vx, vy	0.025	Linear velocities
vz	0.04	Vertical velocity
vroll, vpitch	0.01	Angular rates
vyaw	0.02	Yaw rate

2.3 Results

EKF Performance vs Ground Truth

Metric	Value	Unit	Interpretation
RMSE	0.0091 - 0.0106	m	~1 cm average error
ATE (Absolute Trajectory Error)	0.0073 - 0.0091	m	Excellent tracking
RPE (Relative Pose Error)	0.0023	m	Minimal drift
Maximum Error	0.0295	m	Peak deviation ~3 cm
Standard Deviation	0.0054	m	Consistent performance

Odometry Comparison

Source	Mean Error	Max Error	Notes
Raw Wheel Odometry	~5-10 cm	>50 cm	Significant drift over time
EKF Fused	<1 cm	~3 cm	10x improvement

Key Finding: EKF sensor fusion reduced wheel odometry drift by approximately 90-99%.

3. Step 2: Depth to PointCloud2 Conversion

3.1 Objective

Convert RGBD camera depth data to PointCloud2 messages for 3D SLAM.

3.2 Implementation

The Gazebo RGBD camera plugin automatically publishes PointCloud2:

URDF Configuration (p3dx_hw2.urdf.xacro):

```

<gazebo reference="camera_link">
  <sensor type="depth" name="rgbd_camera">
    <update_rate>30.0</update_rate>
    <camera>
      <horizontal_fov>1.5708</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
        <format>R8G8B8</format>
      </image>
      <clip>
        <near>0.1</near>
        <far>4.0</far>
      </clip>
    </camera>
    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
      <ros>
        <namespace>/camera</namespace>
      </ros>
      <hack_baseline>0.07</hack_baseline>
    </plugin>
  </sensor>
</gazebo>

```

3.3 Published Topics

Topic	Message Type	Rate	Description
/camera/rgbd_camera/rgb_image	sensor_msgs/Image	30 Hz	RGB color image
/camera/rgbd_camera/depth_image_raw	sensor_msgs/Image	30 Hz	32-bit float depth
/camera/depth/pointcloud	sensor_msgs/PointCloud2	30 Hz	3D point cloud
/camera/rgbd_camera/camera_info	sensor_msgs/CameraInfo	30 Hz	Camera intrinsics

3.4 Additional Conversion: Depth to LaserScan

For 2D navigation compatibility:

```

# depthimage_to_laserscan configuration
Node(
  package='depthimage_to_laserscan',
  executable='depthimage_to_laserscan_node',
  parameters=[{
    'scan_time': 0.033,      # 30 Hz
    'range_min': 0.1,        # 10 cm
    'range_max': 4.0,        # 4 m
    'scan_height': 60,       # Vertical pixels
    'output_frame_id': 'camera_link',
  }],
)

```

4. Step 3: 3D SLAM - Visual Mode

4.1 Objective

Implement 3D SLAM using visual features from RGB-D camera.

4.2 RTAB-Map Visual SLAM Configuration

Configuration File: `src/robot_project/config/rtabmap_rgbd.yaml`

Feature Detection

Parameter	Value	Description
Kp/DetectorStrategy	6	GFTT (Good Features To Track)
Kp/MaxFeatures	400-500	Features per frame
Kp/MaxDepth	4.0 m	Feature depth limit
Rtabmap/DetectionRate	1.0-2.0 Hz	Loop closure detection

Loop Closure Parameters

Parameter	Value	Description
Rtabmap/LoopThr	0.11	Similarity threshold
Vis/MinInliers	20	Minimum visual matches
RGBD/OptimizeMaxError	1.0	Reject bad closures
RGBD/LoopClosureReextractFeatures	true	Re-verify matches

Graph Optimization

Parameter	Value	Description
Optimizer/Strategy	1	g2o optimizer
Optimizer/Iterations	50	Optimization steps
Optimizer/Slam2D	true	2D constraint
RGBD/LinearUpdate	0.1 m	Min motion for node
RGBD/AngularUpdate	0.1 rad	Min rotation for node

Occupancy Grid

Parameter	Value	Description
Grid/CellSize	0.05 m	5 cm resolution
Grid/RangeMin	0.2 m	Minimum range
Grid/RangeMax	4.0 m	Maximum range
Grid/MaxObstacleHeight	2.0 m	Obstacle threshold
Grid/DepthDecimation	6-8	Point decimation

4.3 Visual SLAM Results

Map Quality Metrics

Metric	Value	Unit
Total Points	1,265,586	points
3D Density	204.2	pts/m ³
2D Density	1,230.3	pts/m ²
Coverage Area	1,028.72	m ²
Bounding Box	37.1 x 27.7 x 6.0	m
Mapping Duration	13.3	minutes

Localization Accuracy

Metric	Value	Unit
RMSE	0.0991	m
ATE	0.0876	m
RPE	0.0156	m
Maximum Error	0.1523	m

5. Step 4: 3D SLAM - ICP Mode

5.1 Objective

Implement 3D SLAM using geometric point cloud matching (ICP).

5.2 RTAB-Map ICP Configuration

Configuration File: `src/robot_project/config/rtabmap_icp.yaml`

ICP Parameters

Parameter	Value	Description
Reg/Strategy	1	ICP registration
Icp/Strategy	1	Point-to-Plane ICP
Icp/VoxelSize	0.05 m	5 cm voxels
Icp/MaxCorrespondenceDistance	0.1 m	10 cm max match
Icp/Iterations	30	ICP iterations
Icp/MaxTranslation	0.3 m	Max translation
Icp/MaxRotation	0.78 rad	45° max rotation
Icp/CorrespondenceRatio	0.3	Match ratio
Icp/OutlierRatio	0.85	Outlier filtering
Icp/Epsilon	0.001	Convergence threshold

Optimization

Parameter	Value	Description
Optimizer/Iterations	100	More iterations than Visual
RGBD/LinearUpdate	0.1 m	Finer updates
RGBD/AngularUpdate	0.1 rad	Finer angular

5.3 ICP SLAM Results

Map Quality Metrics

Metric	Value	Unit
Total Points	82,314	points
3D Density	223.4	pts/m ³
2D Density	492.1	pts/m ²
Coverage Area	167.26	m ²
Bounding Box	18.68 x 8.96 x 2.20	m

Metric	Value	Unit
Mapping Duration	1.0	minutes

Localization Accuracy

Metric	Value	Unit
RMSE	0.0945	m
ATE	0.0834	m
RPE	0.0148	m
Maximum Error	0.1456	m

6. Step 5: Localization Performance Comparison

6.1 Objective

Compare localization accuracy of different methods against Gazebo ground truth.

6.2 Evaluation Methodology

Implementation: `src/robot_project/robot_project/evaluation_node.py`

Metrics Calculated

Metric	Formula	Description
RMSE	$\sqrt{(\Sigma e^2/n)}$	Root Mean Square Error
ATE	$\Sigma e /n$	Absolute Trajectory Error
RPE	Relative motion error	Relative Pose Error (per 1s)
Max Error	$\max(e)$	Maximum deviation
Std Dev	(e)	Error consistency

Comparison Sources

1. **Ground Truth:** `/ground_truth/odom` (Gazebo p3d plugin, perfect)
2. **EKF Filtered:** `/odometry/filtered` (sensor fusion output)
3. **SLAM Pose:** `/localization_pose` (RTAB-Map corrected)

6.3 Quantitative Comparison

EKF vs Ground Truth

Metric	Best Run	Typical	Worst	Unit
RMSE	0.0012	0.0091	0.0147	m
ATE	0.0012	0.0073	0.0118	m
RPE	0.0008	0.0023	0.0035	m
Max Error	0.0089	0.0295	0.0450	m

SLAM Methods vs Ground Truth

Method	RMSE (m)	ATE (m)	RPE (m)	Max Error (m)
EKF Only	0.0091	0.0073	0.0023	0.0295
Visual SLAM	0.0991	0.0876	0.0156	0.1523
ICP SLAM	0.0945	0.0834	0.0148	0.1456

Improvement Analysis

Comparison	Improvement
ICP vs Visual RMSE	4.6% better
ICP vs Visual ATE	4.8% better
ICP vs Visual RPE	5.1% better
EKF vs SLAM	~10x better (expected)

6.4 Qualitative Analysis

Visual SLAM Characteristics

- **Pros:** Rich feature matching, good in textured environments
- **Cons:** Sensitive to lighting, texture-dependent
- **Best For:** Office environments with distinct visual features

ICP SLAM Characteristics

- **Pros:** Texture-independent, robust geometric matching
- **Cons:** Higher computational cost, needs good geometry
- **Best For:** Structured environments, low-texture areas

Key Findings

1. EKF alone provides excellent short-term accuracy (<3 cm)
2. SLAM adds loop closure capability but introduces mapping uncertainty
3. ICP slightly outperforms Visual SLAM in this office environment
4. Both SLAM methods achieve <10 cm average error

7. Step 6: 3D Mapping Performance Comparison

7.1 Objective

Compare 3D mapping quality between Visual and ICP SLAM methods.

7.2 Evaluation Methodology

Implementation: `src/robot_project/robot_project/map_metrics.py`

Metrics Calculated

Metric	Description	Unit
Point Count	Total 3D points	points
3D Density	Points per cubic meter	pts/m ³
2D Density	Points per square meter	pts/m ²
Coverage	2D footprint area	m ²
Volume	3D bounding box volume	m ³

Metric	Description	Unit
Bounding Box	X, Y, Z dimensions	m

7.3 Quantitative Comparison

Map Size and Coverage

Metric	Visual SLAM	ICP SLAM	Difference
Total Points	1,265,586	82,314	15.4x more (Visual)
Coverage Area	1,028.72 m ²	167.26 m ²	6.1x more (Visual)
Volume	6,196.82 m ³	368.47 m ³	16.8x more (Visual)
Duration	13.3 min	1.0 min	13.3x longer (Visual)

Point Cloud Density

Metric	Visual SLAM	ICP SLAM	Winner
3D Density	204.2 pts/m ³	223.4 pts/m ³	ICP (+9.4%)
2D Density	1,230.3 pts/m ²	492.1 pts/m ²	Visual (+150%)

Bounding Box Dimensions

Dimension	Visual SLAM	ICP SLAM
X	37.1 m	18.68 m
Y	27.7 m	8.96 m
Z	6.0 m	2.20 m

7.4 Mapping Progression Over Time (Visual SLAM)

Time (s)	Points	Density (pts/m ³)	Coverage (m ²)
60	~50,000	150	~50
180	~200,000	175	~200
360	~500,000	190	~500
540	~800,000	200	~800
796	1,265,586	204.2	1,028.72

Observation: Point density stabilized around 200 pts/m³ after 6 minutes.

7.5 2D Occupancy Grid Output

Parameter	Value
Format	PGM (Portable GrayMap)
Resolution	0.05 m/pixel (5 cm)
Size	138 x 115 pixels
Coverage	6.9 x 5.75 m
Occupied Threshold	0.65
Free Threshold	0.25

7.6 Qualitative Analysis

Visual SLAM Mapping

- **Dense point clouds** with rich color information
- **Higher 2D coverage** due to feature-based exploration
- **Better for navigation** - more complete occupancy grid
- **Slower** due to feature extraction overhead

ICP SLAM Mapping

- **Higher 3D density** per cubic meter
- **Faster processing** - pure geometric matching
- **More consistent** geometry reconstruction
- **Limited by sensor range** (4m depth camera)

Recommendation

- Use **Visual SLAM** for comprehensive mapping and navigation
 - Use **ICP SLAM** for quick geometric verification
-

8. Step 7: Autonomous Navigation with Nav2

8.1 Objective

Use 2D projection of 3D map for autonomous navigation with random waypoints.

8.2 System Components

8.2.1 Hybrid SLAM Controller

File: `src/robot_project/robot_project/hybrid_slam_controller.py`

Provides manual/auto control during SLAM mapping:

Mode	Description
MANUAL	Direct WASD control, no obstacle avoidance
AUTO	Depth-based autonomous exploration
TURBO	AUTO with 2x speed multiplier

Keyboard Controls

Key	Action
WASD	Movement control
1-5	Speed levels (0.2x - 2.0x)
Q/E	90° turns
R	180° U-turn
SPACE	Toggle AUTO/MANUAL
T	Toggle TURBO
P	Pause (for Nav2 control)
ESC	Emergency stop

5-Region Obstacle Detection

FAR LEFT	LEFT	CENTER	RIGHT	FAR RIGHT
0-15%	15-35%	35-65%	65-85%	85-100%

8.2.2 Nav2 Configuration

File: `src/robot_project/config/nav2_params.yaml`

Robot Footprint

Parameter	Value
Robot Radius	0.22 m
Inflation Radius	0.35 m
Cost Scaling Factor	3.0

Velocity Limits

Parameter	Value
Max Linear	1.2 m/s
Max Angular	2.0 rad/s
Linear Acceleration	2.5 m/s ²
Angular Acceleration	4.0 rad/s ²

Costmap Configuration

Parameter	Local	Global
Update Frequency	10 Hz	2 Hz
Resolution	0.05 m	0.05 m
Rolling Window	4x4 m	Full map

Goal Tolerances

Parameter	Value
XY Tolerance	0.25 m
Yaw Tolerance	0.25 rad (~14°)

8.3 Navigation Workflow

NAVIGATION WORKFLOW

1. SLAM Phase (Hybrid Controller)

```
slam_hybrid    3D Map + 2D Grid
.launch.py
```

Press P to pause controller

2. Nav2 Activation (Separate Terminal)

```
nav2_bringup    Planner + Controller active
navigation
```

3. Goal Selection (RViz)

```
2D Goal Pose    /goal_pose topic
tool
```

4. Autonomous Execution

```
Global Path      A* on /map
Local Path       DWB obstacle avoidance
Recovery         Spin, backup behaviors
```

8.4 Launch Commands

Terminal 1: SLAM with Hybrid Controller

```
ros2 launch robot_project slam_hybrid.launch.py
```

Terminal 2: Nav2 (after SLAM starts ~30s)

```
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true
```

In RViz: Use "2D Goal Pose" to set navigation goals

8.5 Navigation Results

Successful Navigation Metrics

Metric	Value
Goal Reach Success Rate	>90%
Average Path Length	Optimal (A* based)
Obstacle Avoidance	Real-time (DWB)
Recovery Behaviors	Spin, Backup, Wait

Integration with SLAM

- 2D occupancy grid from RTAB-Map directly used by Nav2
 - Real-time map updates during exploration
 - Loop closure corrections propagate to navigation
-

9. Implementation Challenges and Solutions

9.1 Robot Physics Issues

Problem: Robot Falling Through Ground

Cause: Missing ground plane in Gazebo world file **Solution:** Added ground_plane model to empty_office.world

```
<model name='ground_plane'>
  <static>true</static>
  <link name='link'>
    <collision name='collision'>
      <geometry>
        <plane><normal>0 0 1</normal><size>50 50</size></plane>
      </geometry>
      <surface>
        <friction><ode><mu>100</mu><mu2>50</mu2></ode></friction>
      </surface>
    </collision>
  </link>
</model>
```

9.2 SLAM Stability Issues

Problem: Map Drift and Reset During SLAM

Cause: False loop closure detections in similar-looking office areas **Solution:** Added strict loop closure parameters

```
'Vis/MinInliers': '20',           # Minimum visual matches
'RGBD/OptimizeMaxError': '1.0',    # Reject bad closures
'Rtabmap/LoopThr': '0.11',         # Higher threshold
'RGBD/LoopClosureReextractFeatures': 'true', # Re-verify
```

9.3 Controller Conflicts

Problem: Hybrid Controller Conflicts with Nav2

Cause: Both writing to /cmd_vel simultaneously **Solution:** Pause controller before using Nav2 1. Press **P** to pause Hybrid Controller 2. Set goal in RViz with “2D Goal Pose” 3. Nav2 controls robot 4. Press **P** to resume manual control

9.4 Navigation Planning Failures

Problem: “Failed to create plan” Errors

Cause: Robot position in unknown/occupied costmap area **Solutions:** 1. Ensure /map topic is publishing 2. Clear costmaps: `ros2 service call /global_costmap/clear_entirely_global_costmap` 3. Verify TF tree: `ros2 run tf2_tools view_frames`

9.5 Database Corruption

Problem: RTAB-Map database corrupted after system crash

Cause: Copying database with cp during active SLAM **Solution:** Use SQLite backup API for safe copies

```
import sqlite3
source = sqlite3.connect('~/.ros/rtabmap.db')
dest = sqlite3.connect('backup.db')
source.backup(dest) # Transaction-safe
```


10. Conclusions

10.1 Requirements Completion Summary

Step	Requirement	Status	Key Result
1	EKF Sensor Fusion		<1 cm RMSE, 10x improvement over raw odometry
2	Depth to PointCloud2		30 Hz, 640x480, 0.1-4.0m range
3	Visual SLAM		1.2M points, 1028 m ² coverage
4	ICP SLAM		Higher 3D density (223 pts/m ³)
5	Localization Comparison		ICP 4.6% better than Visual
6	Mapping Comparison		Visual 15x more points, ICP denser
7	Autonomous Navigation		Nav2 integration, goal-based navigation

10.2 Key Findings

1. **EKF Sensor Fusion** dramatically improves odometry accuracy (90-99% drift reduction)
2. **Visual SLAM** produces larger, more complete maps suitable for navigation
3. **ICP SLAM** achieves slightly better localization accuracy (+4.6%)
4. **Hybrid Control** enables flexible manual/auto exploration during SLAM
5. **Nav2 Integration** successfully uses RTAB-Map 2D grid for autonomous navigation

10.3 Future Improvements

1. Implement multi-session mapping with map merging
2. Add dynamic obstacle detection and tracking
3. Integrate semantic segmentation for room-based navigation
4. Optimize for larger environments (>1000 m²)

Appendix A: File Locations

Resource	Path
EKF Config	src/robot_project/config/robot_localization.yaml
RTAB-Map Visual	src/robot_project/config/rtabmap_rgbd.yaml
RTAB-Map ICP	src/robot_project/config/rtabmap_icp.yaml
Nav2 Config	src/robot_project/config/nav2_params.yaml
Hybrid Controller	src/robot_project/robot_project/hybrid_slam_controller.py
Map Metrics	src/robot_project/robot_project/map_metrics.py
Evaluation Node	src/robot_project/robot_project/evaluation_node.py
Main Launch	src/robot_project/launch/slam_hybrid.launch.py
Robot URDF	src/robot_hw1/urdf/p3dx_hw2.urdf.xacro
World File	src/robot_hw1/worlds/empty_office.world
Results Data	project/results/data/
RTAB-Map Database	~/ros/rtabmap.db

Appendix B: Commands Reference

```
# Build
cd /home/mmf/Documents/GitHub/hws_repo
source /opt/ros/humble/setup.bash
```

```
colcon build --symlink-install
source install/setup.bash

# SLAM with Hybrid Controller
ros2 launch robot_project slam_hybrid.launch.py

# Nav2 Navigation (separate terminal)
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true

# Monitoring
ros2 topic hz /map
ros2 node list | grep rtabmap
ros2 run tf2_tools view_frames

# Database Backup
cp ~/.ros/rtabmap.db ~/maps/backup_$(date +%Y%m%d_%H%M%S).db
```

Appendix C: Data Collection Summary

Data Type	Files	Samples	Description
Position Metrics	467	~500K	EKF/SLAM RMSE, ATE, RPE
Map Metrics	608	~700K	Point count, density, coverage
Ground Truth	42	~200K	Perfect trajectories
Filtered Odometry	42	~200K	EKF output trajectories

Total Data Points: ~1.6 million measurements

Report generated: December 2024 Repository: <https://github.com/mmf-code/hws>